**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**

Факультет прикладной математики–процессов управления

Программа бакалавриата

**«Большие данные и распределенная цифровая платформа»**

# ОТЧЕТ

По проектной работе

По дисциплине «Алгоритмы и структуры данных»

На тему «Распознавание рукописаного текста и анализ эмоциональной окраски»

Студенты гр. 24Б15-пу
Телицин А.Д.

Шаймиев К.А.

Преподаватель
Дик А.Г.

Санкт-Петербург
2026 г.

# Содержание

**1 Цель работы** **2**
1.1 Задачи . . . 2

**2 Теоретическая часть** **2**
2.1 Распознавание текста и предобработка . . . 2
2.2 Нейросетевая архитектура OCR . . . 3
2.3 Датасеты и оценка качества . . . 4
2.4 Постобработка и анализ эмоций . . . 4

**3 Используемые методы и алгоритмы, их пошаговое описание** **4**
3.1 Общая архитектура программы . . . 4
3.1.1 Загрузка датасета . . . 5
3.1.2 Предобработка изображений . . . 5
3.1.3 Обучение CRNN-модели . . . 6
3.1.4 Применение обученной модели к изображениям . . . 6
3.1.5 Постобработка текста и анализ эмоций . . . 6
3.2 Используемые алгоритмы . . . 7
3.2.1 CRNN (сверточная-рекуррентная нейросеть) . . . 7
3.2.2 CTC-алгоритм . . . 7
3.2.3 Триграммное сходство . . . 8
3.2.4 Лемматизация с использованием pymorphy3 . . . 8
3.2.5 Подсчет распределения эмоций . . . 8

**4 Блок-схемы и диаграмма компонентов** **9**
4.1 Диаграмма компонентов . . . 9
4.2 Блок-схемы основных компонентов . . . 10
4.2.1 Загрузка датасета . . . 10
4.2.2 Предобработка изображений . . . 11
4.2.3 Обучение CRNN . . . 12
4.2.4 Применение обученной модели к изображениям . . . 13
4.2.5 Постобработка распознанного текста . . . 14
4.2.6 Анализ эмоций . . . 15

**5 Рекомендации пользователю** **15**
5.1 Описание элементов интерфейса . . . 16
5.2 Интерпретация результатов . . . 16
5.3 Рекомендации по использованию . . . 17

**6 Рекомендации программисту и дополнительные сведения о ПО** **17**

# 1 Цель работы

Целью работы является разработка программного инструмента для автоматического распознавания рукописаного текста и последующего анализа тональности распознанного содержания.

## 1.1 Задачи

Для достижения данной цели были поставлены следующие задачи:

1. Проанализировать существующие подходы к распознаванию рукописного текста и анализу тональности.
2. Подобрать и подготовить датасет для обучения и тестирования модели.
3. Определить метрики качества и критерии успешности решения.
4. Разработать и реализовать модель распознавания рукописного текста.

# 2 Теоретическая часть

В этом разделе приведены ключевые термины и методы, используемые в проекте распознавания рукописного текста и анализа эмоциональной окраски. Далее они будут использоваться для описания архитектуры модели, процесса обучения и оценки качества.

## 2.1 Распознавание текста и предобработка

**Оптическое распознавание символов (OCR, Optical Character Recognition)** Автоматическое преобразование изображения с текстом в цифровую строку. *В проекте:* используется для распознавания рукописных записей на русском и английском языках.

**Распознавание рукописного текста (HTR, Handwritten Text Recognition)** Частный случай OCR для рукописного ввода, более сложный из-за вариативности почерка. *В проекте:* основная задача модели.

**Предобработка изображения (image preprocessing)** Набор операций перед подачей в модель: снижение шума, выравнивание контраста, нормализация масштаба. *В проекте:* оттенки серого, обрезка по содержимому, изменение размера, нормализация, бинаризация и удаление шума.

**Изображение в оттенках серого (grayscale image)** Формат, где пиксель хранит только яркость. *В проекте:* цвет несущественен, важна форма штрихов.

**Нормализация изображения (image normalization)** Приведение значений пикселей к единому диапазону. *В проекте:* стабилизирует вход нейросети.

**Обрезка по содержимому (content crop)** Удаление пустых полей вокруг текста. *В проекте:* повышает долю полезной информации во входе модели.

**Бинаризация (binarization)** Преобразование изображения в черно-белый вид по порогу. *В проекте:* используется как дополнительный режим, может как улучшать, так и ухудшать качество при неверном пороге.

**Связная компонента (connected component)** Группа соседних пикселей в бинарном изображении. *В проекте:* мелкие компоненты удаляются как шум.

**Сегментация (segmentation)** Разбиение изображения на строки/слова/фрагменты. *В проекте:* применяется для распознавания частей строки с последующей сборкой результата.

**Профиль чернил (ink profile)** Распределение темных пикселей по строкам или столбцам. *В проекте:* используется для поиска границ строк и слов без отдельной модели сегментации.

### 2.2 Нейросетевая архитектура OCR

**Сверточная нейронная сеть (CNN, Convolutional Neural Network)** Извлекает визуальные признаки: линии, углы, контуры. *В проекте:* формирует признаки рукописных символов.

**Сверточный слой (convolutional layer)** Применяет фильтры к изображению/картам признаков. *В проекте:* выделяет локальные элементы букв.

**Пулинг (pooling)** Уменьшает размер карты признаков с сохранением важной информации. *В проекте:* снижает размерность и повышает устойчивость к сдвигам.

**Рекуррентная нейронная сеть (RNN, Recurrent Neural Network)** Обрабатывает последовательности с учетом контекста. *В проекте:* работает с последовательностью признаков вдоль строки.

**Долгая краткосрочная память (LSTM, Long Short-Term Memory)** Тип RNN, лучше сохраняющий долгий контекст. *В проекте:* помогает различать похожие символы по соседним.

**Двунаправленная LSTM (BiLSTM, Bidirectional LSTM)** Анализирует последовательность слева направо и справа налево. *В проекте:* учитывает контекст с обеих сторон.

**Сверточно-рекуррентная сеть (CRNN, Convolutional Recurrent Neural Network)** Комбинация CNN и RNN для распознавания текста с изображения. *В проекте:* основная архитектура OCR.

**Тензор (tensor)** Многомерный числовой массив. *В проекте:* стандартный формат входа/выхода в PyTorch.

**CTC-функция потерь (CTC-loss, Connectionist Temporal Classification loss)** Позволяет обучать модель по парам “изображение–строка” без посимвольной разметки. *В проекте:* ключевой механизм обучения CRNN.

**Декодирование CTC (CTC decoding)** Преобразует последовательность вероятностей в текст (с удалением повторов и пустых символов). *В проекте:* финальный этап OCR-предсказания.

**Инференс (inference)** Процесс использования обученной модели для предсказания на новых данных. *В проекте:* применяется для получения результатов распознавания.

### 2.3 Датасеты и оценка качества

**IAM Handwriting Database** [1] Набор рукописного английского текста с аннотациями. *В проекте:* используется для английского OCR-пайплайна.

**HKR/HTR dataset** [2] Набор рукописного русского текста (изображения + JSON-аннотации). *В проекте:* основной датасет для русского OCR.

**Обучающая выборка (training set)** Данные для обновления параметров модели.

**Валидационная выборка (validation set)** Данные для контроля качества без обновления весов.

**Checkpoint** Файл сохраненного состояния модели. *В проекте:* позволяет запускать инференс без повторного обучения.

**CER (Character Error Rate)** Доля ошибок на уровне символов.

**WER (Word Error Rate)** Доля ошибок на уровне слов.

**Расстояние Левенштейна (Levenshtein distance)** Минимальное число вставок, удалений и замен для преобразования одной строки в другую. *В проекте:* используется при вычислении CER и WER.

### 2.4 Постобработка и анализ эмоций

**Постобработка текста (text postprocessing)** Очистка и нормализация OCR-результата перед дальнейшим анализом.

**Токенизация (tokenization)** Разбиение текста на токены (обычно слова).

**Лемматизация (lemmatization)** Приведение слов к начальной форме.

**pymorphy3 morphological analyzer** Морфологический анализатор русского языка. *В проекте:* применяется для лемматизации.

**Коррекция ошибок OCR (OCR error correction)** Исправление слов, распознанных с ошибками.

**Триграммное сходство (trigram similarity)** Сравнение слов по совпадению трехсимвольных подстрок. *В проекте:* используется при подборе корректировок.

**Лексиконный анализ эмоций (lexicon-based emotion analysis)** Определение эмоций по словарю с заранее размеченными словами.

**RusEmoLex** Русскоязычный эмоциональный словарь. *В проекте:* источник эмоциональных меток для слов.

## 3 Используемые методы и алгоритмы, их пошаговое описание

### 3.1 Общая архитектура программы

Программа состоит из нескольких основных компонентов:

- **Загрузка датасета:** чтение изображений и аннотаций из выбранного датасета.

- **Предобработка изображения:** преобразование в оттенки серого, повышение контрастности, обрезка по содержимому, изменение размера, нормализация, бинаризация и удаление шума. Приведение к матричному виду для подачи в модель.
- **Обучение CRNN-модели:** сверточно-рекуррентная нейросеть для распознавания текста.
- **Применение обученной модели к изображениям:** преобразование выходов модели в текстовую строку.
- **Постобработка текста (для русского):** токенизация, коррекция OCR-ошибок и лемматизация.
- **Анализ эмоций:** поиск слов в RusEmoLex и вычисление распределения эмоций.

### 3.1.1 Загрузка датасета

В качестве основного источника данных для русского языка используется HKR/HTR dataset, который содержит изображения рукописного текста и соответствующие JSON-аннотации. Для английского языка используется IAM Handwriting Database [1].

Загрузка данных для русского и английского вариантов не имеет принципиальных отличий, так как оба датасета предоставляют изображения и текстовые аннотации. Основные шаги:

1. Чтение списка файлов изображений и соответствующих аннотаций.
2. Сопоставление каждого изображения с его текстовой строкой.
3. Разделение данных на обучающую, валидационную и тестовую выборки.

### 3.1.2 Предобработка изображений

После загрузки изображения проходят через несколько этапов предобработки:

1. **Преобразование в оттенки серого:** удаление цветовой информации для упрощения анализа.
2. **Повышение контрастности:** позволяет повысить различимость штрихов на фоне и удалить артефакты.
3. **Обрезка по содержимому:** удаление пустых полей вокруг текста для фокусировки на полезной информации.
4. **Изменение размера:** приведение к фиксированной высоте (например, 64 пикселя) с сохранением пропорций.
5. **Нормализация:** масштабирование значений пикселей в диапазон [0, 1].
6. **Бинаризация (опционально):** преобразование в черно-белый вид по порогу для удаления шума.
7. **Приведение к матричному виду:** преобразование изображения в формат, удобный для подачи в нейросеть.

### 3.1.3 Обучение CRNN-модели

В работе используется сверточно-рекуррентная нейросеть (CRNN) из библиотеки PyTorch. Обучение модели происходит на обучающей выборке, а качество контролируется на валидационной выборке.

CRNN-модель состоит из нескольких сверточных слоев для извлечения признаков и рекуррентных слоев для обработки последовательности признаков. Обучение происходит следующим образом:

1. Инициализация модели и оптимизатора.
2. Проход по обучающей выборке в пакетах (батчах) заданного размера.
3. Для каждого изображения: предобработка и подача в модель.
4. Получение выходов модели в виде последовательности вероятностей символов.
5. Вычисление CTC-функции потерь между предсказанием и истинной строкой.
6. Обратное распространение ошибки и обновление весов модели.
7. Периодическая оценка на валидационной выборке для контроля качества.

На выходе обучения сохраняется файл, который содержит веса модели (checkpoint), что позволяет использовать ее для инференса без повторного обучения.

### 3.1.4 Применение обученной модели к изображениям

После обучения модель может быть применена к новым изображениям для получения текстовой строки. Процесс инференса включает следующие шаги:

1. Загрузка сохраненного checkpoint модели.
2. Предобработка входного изображения аналогично тому, что использовалось при обучении.
3. Подача изображения в модель и получение последовательности вероятностей символов.
4. Декодирование CTC для преобразования вероятностей в текстовую строку.
5. Если применялась сегментация, объединение результатов фрагментов в одну строку.

### 3.1.5 Постобработка текста и анализ эмоций

Для русского текста после получения строки из OCR применяется постобработка:

1. **Токенизация:** разбиение строки на отдельные слова.
2. **Коррекция OCR-ошибок:** для каждого слова, не найденного в словаре, поиск наиболее похожего по триграммному сходству и замена на него.
3. **Лемматизация:** приведение слов к начальной форме с помощью pymorphy3.
4. **Анализ эмоций:** для каждого лемматизированного слова поиск в RusEmoLex и подсчет распределения эмоций по тексту.

После всех этапов программа выводит несколько вариантов распознавания (с разными режимами предобработки), исправленный текст (для русского) и информацию о доминирующей эмоции и распределении эмоций в тексте.

### 3.2 Используемые алгоритмы

Вышеперечисленные этапы реализованы с помощью следующих алгоритмов и методов:

- **CRNN (сверточная-рекуррентная нейросеть):** сочетание сверточных и рекуррентных слоев для эффективного распознавания текста с изображений.
- **CTC-алгоритм:** позволяет обучать модель без посимвольной разметки, что критично для рукописного текста.
- **Триграммное сходство:** используется для коррекции OCR-ошибок путем сравнения с лексиконом.
- **Лемматизация с pymorphy3:** позволяет привести слова к начальной форме для более точного анализа эмоций.
- **Подсчет распределения эмоций:** суммирование эмоциональных меток из RusEmoLex для определения доминирующей эмоции.

Ниже приведено более подробное описание алгоритмов, используемых на каждом этапе.

#### 3.2.1 CRNN (сверточная-рекуррентная нейросеть)

CRNN объединяет преимущества сверточных нейросетей (CNN) для извлечения признаков из изображений и рекуррентных нейросетей (RNN) для обработки последовательностей. Сверточная нейронная сеть является усовершенствованной версией обычной полносвязной нейронной сети. Она использует сверточные слои для извлечения локальных признаков из изображения, что позволяет эффективно распознавать сложные паттерны, такие как рукописные символы.

Рекуррентная нейронная сеть, в свою очередь, обрабатывает последовательность признаков, учитывая контекст и взаимосвязи между символами. Это особенно важно для рукописного текста, где форма символов может сильно варьироваться в зависимости от соседних символов. В нашем случае архитектура включает:

- Несколько сверточных слоев для извлечения признаков. Каждый слой применяет набор фильтров к входному изображению или карте признаков, выделяя сложные геометрические шаблоны, характерные для рукописного текста.
- Пулинг-слои для снижения размерности.
- Двунаправленные LSTM-слои для обработки последовательности признаков.
- Выходной слой с активацией softmax для получения вероятностей символов.

#### 3.2.2 CTC-алгоритм

CTC (Connectionist Temporal Classification) — это алгоритм, который позволяет обучать модели распознавания текста без необходимости в посимвольной разметке. Он работает следующим образом:

- Модель генерирует последовательность вероятностей для каждого символа (включая специальный символ "пусто") на каждом временном шаге. Вероятности получаеются на выходе сверточно-рекуррентной нейросети после обработки входного изображения.

- CTC-функция потерь вычисляет вероятность правильного распознавания строки, суммируя по всем возможным выравниваниям между входной последовательностью и целевой строкой.
- Во время инференса декодирование CTC удаляет повторяющиеся символы и пустые символы, чтобы получить итоговую строку.

Преимущество CTC заключается в том, что он позволяет обучать модель на данных, где нет точного выравнивания между входом и выходом, что идеально подходит для рукописного текста.

### 3.2.3 Триграммное сходство

Данный алгоритм используется для коррекции ошибок OCR. Он работает следующим образом:

- Для каждого слова, распознанного OCR, которое не найдено в словаре, генерируется множество триграмм (последовательностей из трех символов).
- Для каждого слова в словаре также генерируются триграммы.
- Вычисляется сходство между распознанным словом и каждым словом в словаре на основе количества совпадающих триграмм.
- Слово из словаря с наибольшим сходством выбирается в качестве корректировки для распознанного слова.

### 3.2.4 Лемматизация с использованием pymorphy3

Лемматизация — это процесс приведения слова к его начальной форме (лемме). В нашем проекте для русского языка используется библиотека pymorphy3, которая предоставляет мощные инструменты для морфологического анализа. Процесс лемматизации включает следующие шаги:

- Для каждого слова, полученного после коррекции OCR-ошибок, вызывается функция лемматизации из pymorphy3.
- Библиотека анализирует слово и возвращает его лемму, которая используется для дальнейшего анализа эмоций.

### 3.2.5 Подсчет распределения эмоций

Для анализа эмоциональной окраски текста используется лексикон RusEmoLex, который содержит слова и их эмоциональные метки. Процесс подсчета распределения эмоций включает следующие шаги:

- Для каждого лемматизированного слова выполняется поиск в RusEmoLex.
- Если слово найдено, его эмоциональные метки добавляются к общему счетчику для каждой эмоции.
- После обработки всех слов вычисляется распределение эмоций, и определяется доминирующая эмоция.

# 4 Блок-схемы и диаграмма компонентов

## 4.1 Диаграмма компонентов

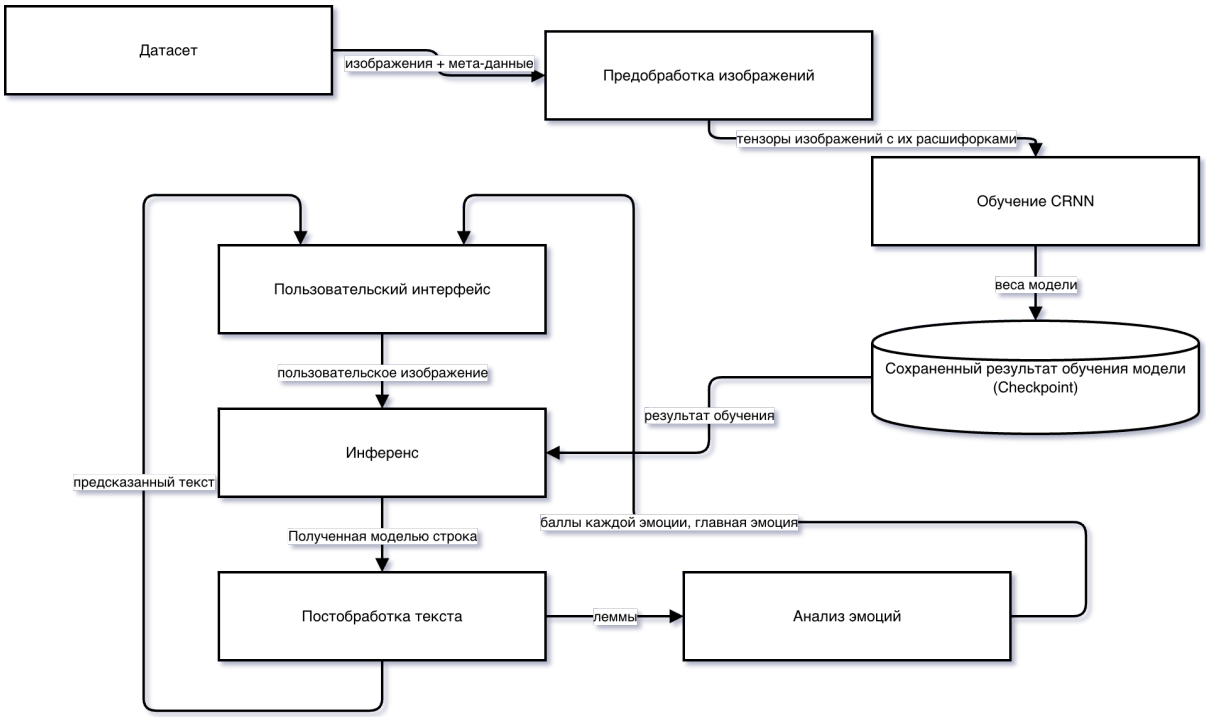


Рис. 1: Диаграмма компонентов программы

## 4.2 Блок-схемы основных компонентов

### 4.2.1 Загрузка датасета

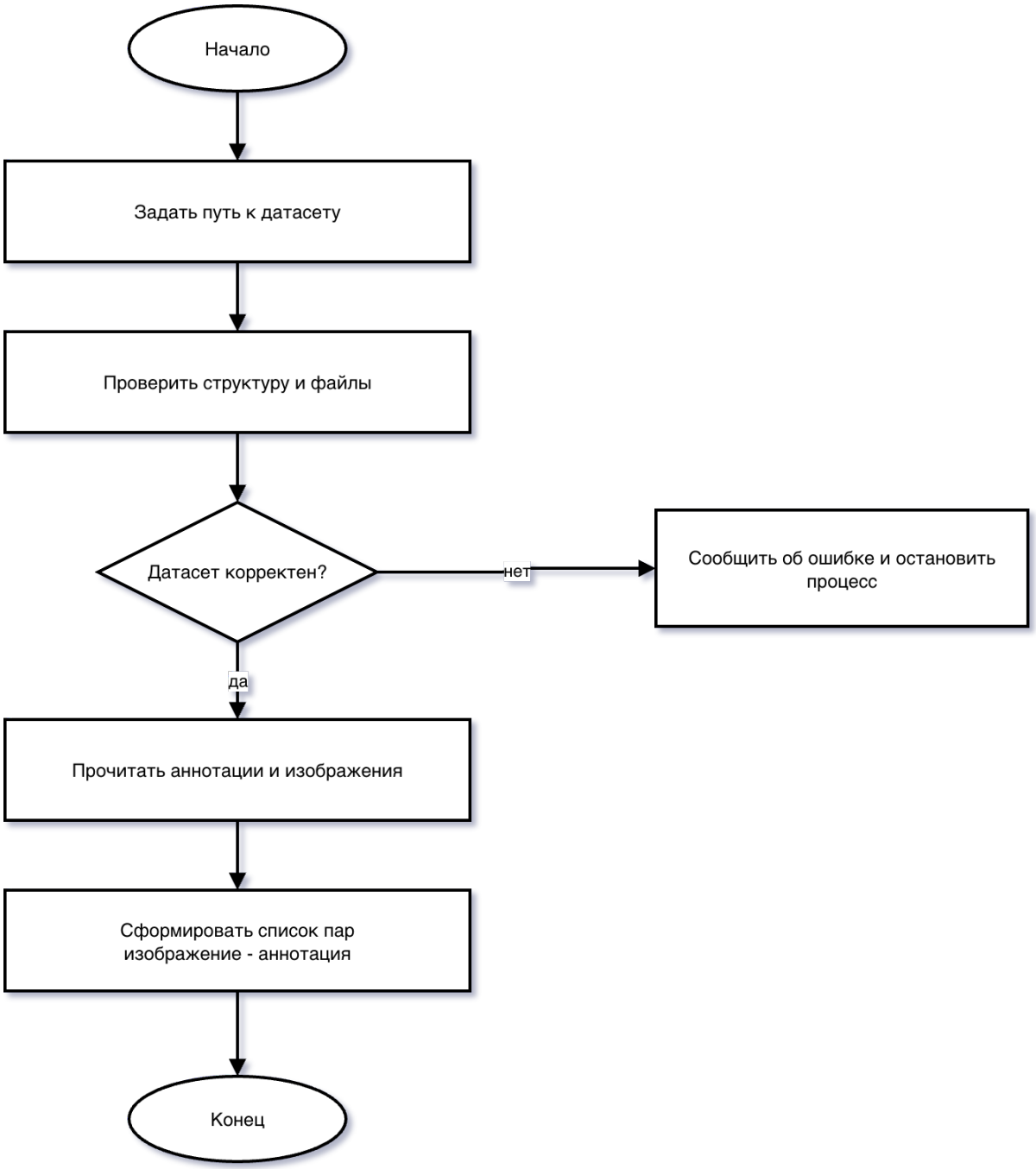


Рис. 2: Блок-схема загрузки датасета

### 4.2.2 Предобработка изображений

Рис. 3: Блок-схема предобработки изображений

### 4.2.3 Обучение CRNN

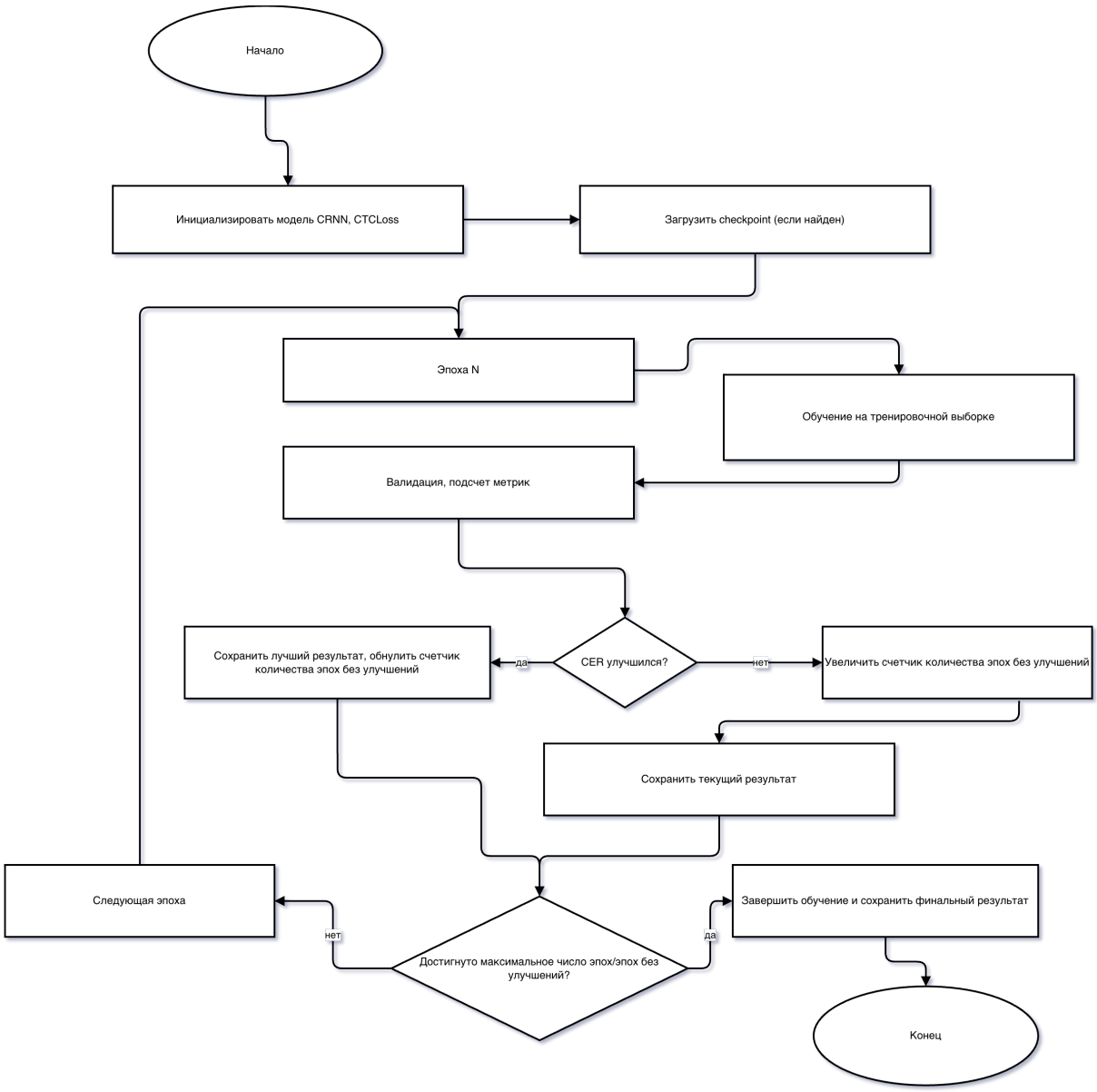


Рис. 4: Блок-схема обучения CRNN-модели

### 4.2.4 Применение обученной модели к изображениям

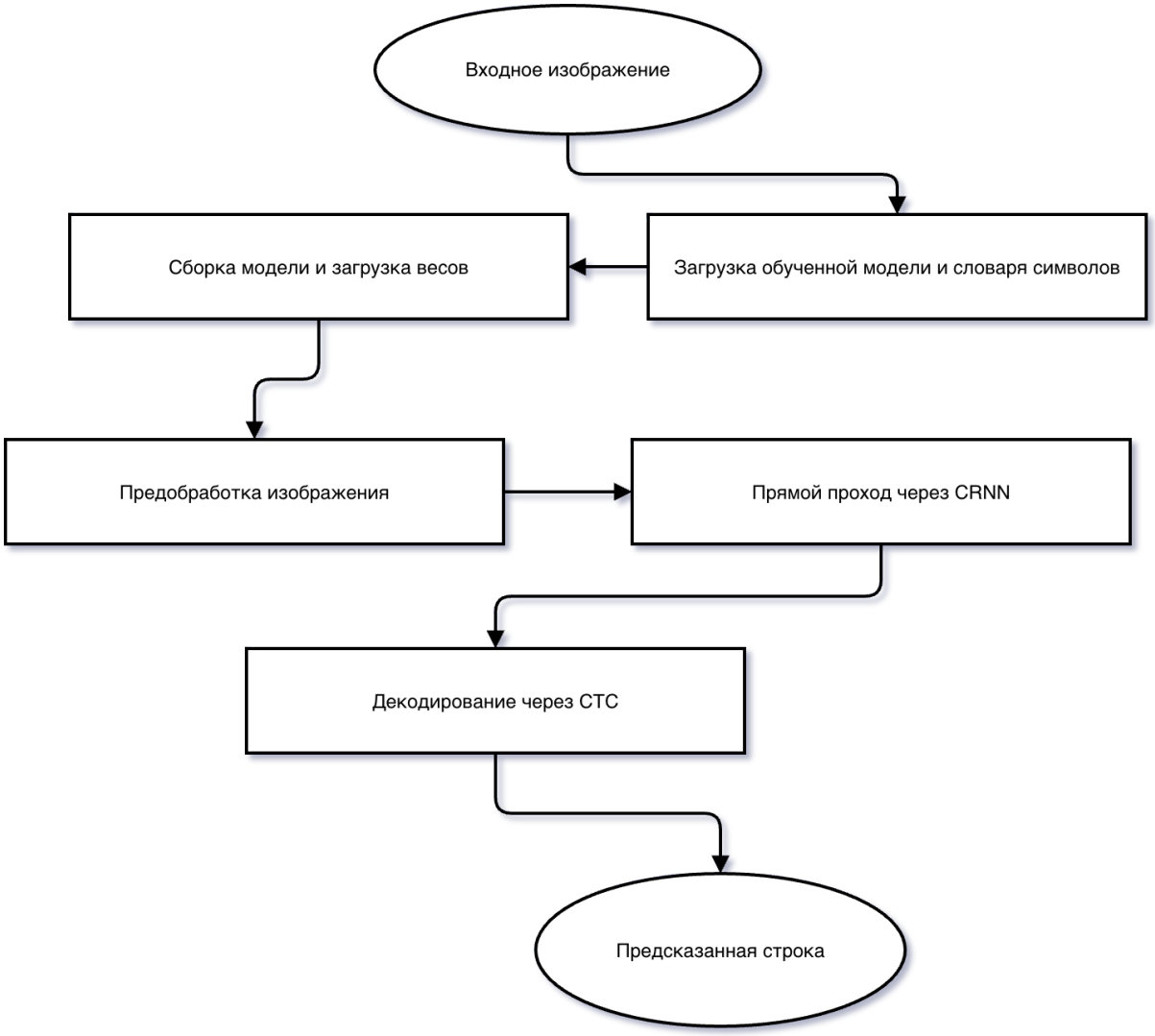


Рис. 5: Блок-схема инференса с обученной CRNN-моделью

### 4.2.5 Постобработка распознанного текста

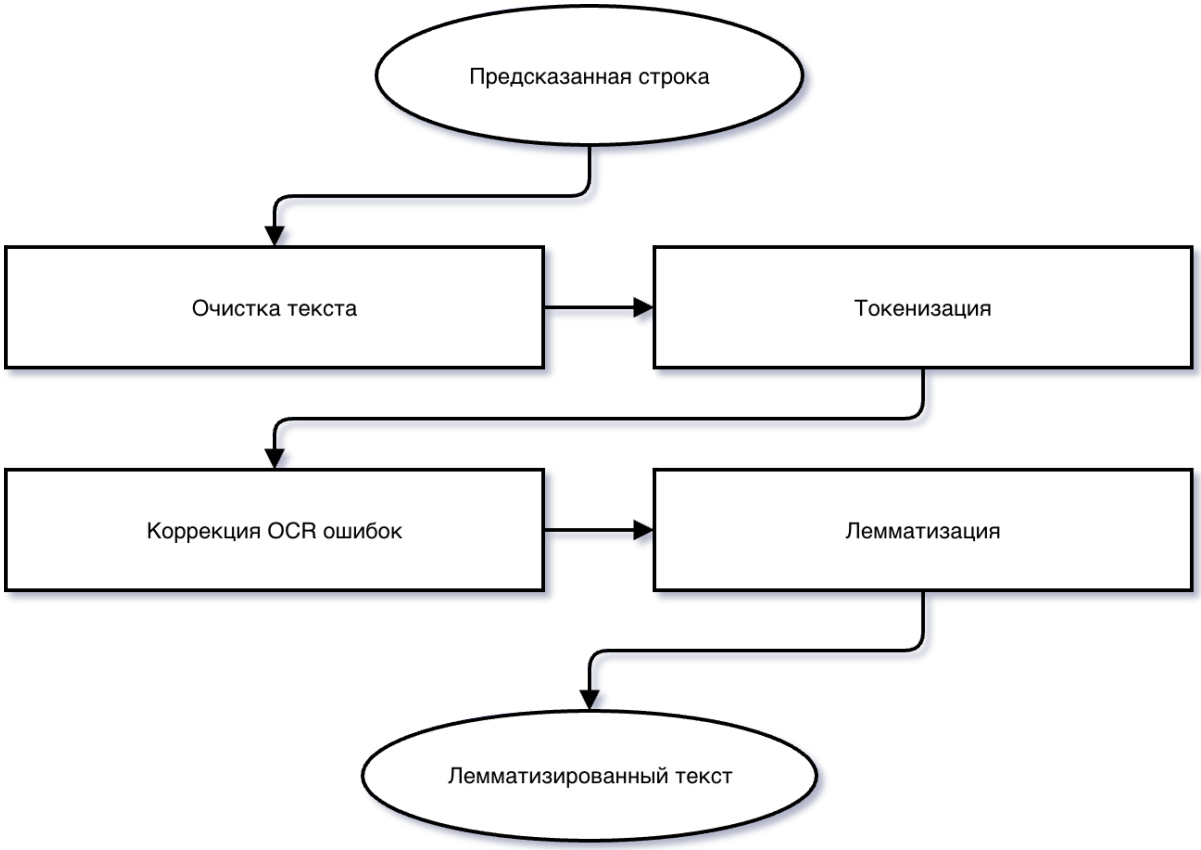


Рис. 6: Блок-схема постобработки распознанного текста

### 4.2.6 Анализ эмоций

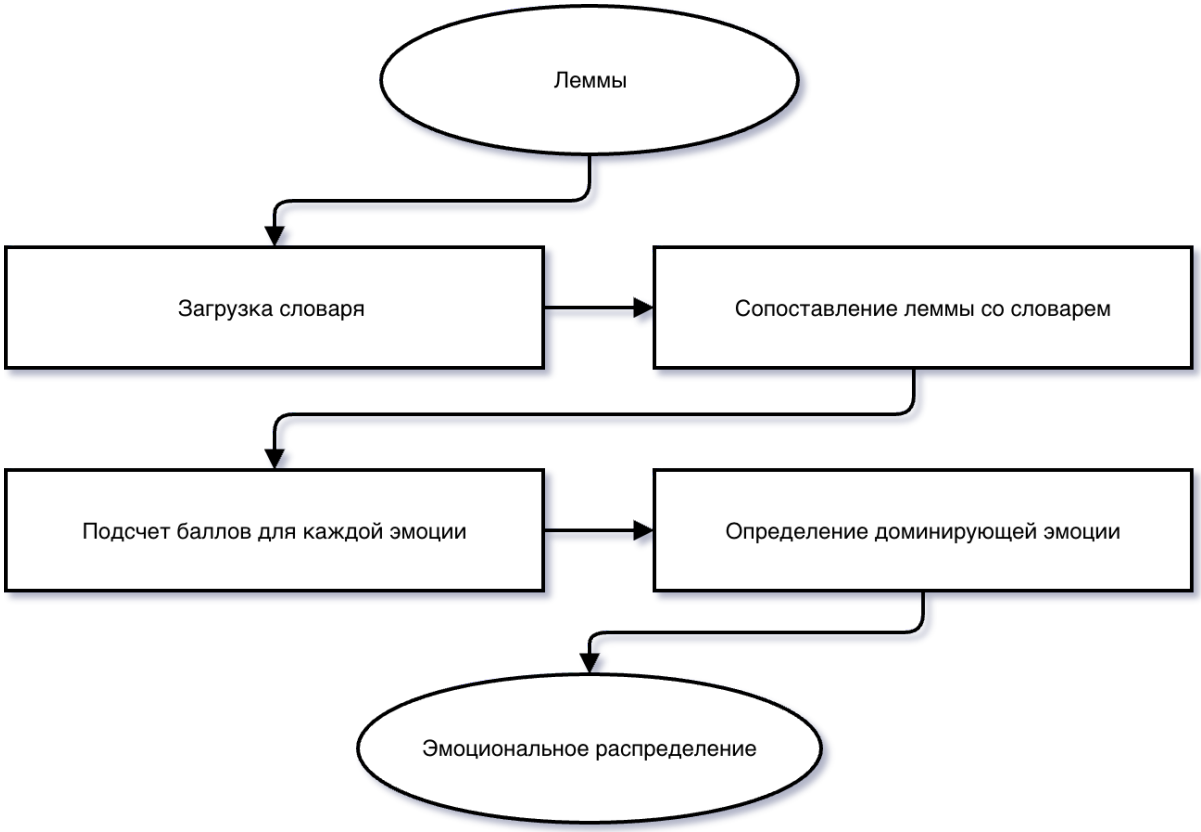


Рис. 7: Блок-схема анализа эмоций

# 5 Рекомендации пользователю

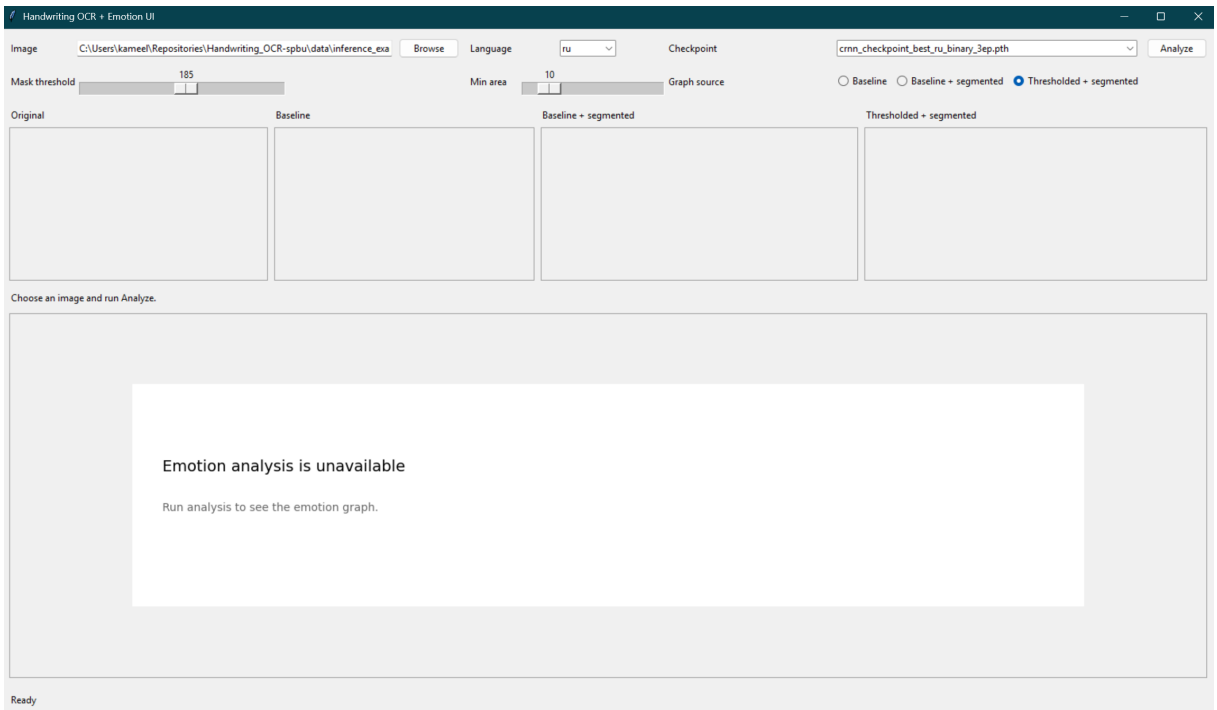


Рис. 8: Пользовательский интерфейс программы

## 5.1 Описание элементов интерфейса

**Поле Image** Путь к изображению, которое будет передано на распознавание. Для выбора файла следует нажать кнопку `Browse`. Если изображение не выбрано или путь указан неверно, программа не сможет выполнить анализ и выведет сообщение об ошибке.

**Поле Language** Язык распознаваемого рукописного текста. Для русскоязычных изображений необходимо выбирать значение `ru`, для англоязычных — `en`. От выбранного языка зависит выбор подходящей модели распознавания и возможность дальнейшего анализа эмоциональной окраски.

**Поле Checkpoint** Файл сохраненного состояния модели, который будет использован для распознавания. Рекомендуется выбирать checkpoint, соответствующий выбранному языку. Для русского текста следует использовать русскую модель, для английского — английскую. Неверно выбранный checkpoint может привести к ухудшению качества распознавания.

**Кнопка Analyze** Кнопка запускает полный цикл обработки: загрузку изображения, предобработку, распознавание текста, построение вариантов результата и, для русского текста, анализ эмоциональной окраски. Перед нажатием необходимо убедиться, что выбраны файл изображения, язык и checkpoint.

**Параметр Mask threshold** Порог бинаризации изображения. При увеличении значения больше пикселей будет относиться к фону, при уменьшении — к рукописному тексту. Если текст на изображении плохо отделяется от бумаги, параметр можно изменить и повторно запустить анализ.

**Параметр Min area** Минимальный размер связной компоненты, которая считается значимой. Мелкие компоненты меньше указанного значения удаляются как шум. Если после обработки пропадают части букв, значение следует уменьшить. Если остается много мелких пятен, значение можно увеличить.

**Переключатель Graph source** Вариант распознавания, по которому строится график эмоционального анализа:

- `Baseline` — распознавание изображения целиком;
- `Baseline + segmented` — распознавание с дополнительной сегментацией;
- `Thresholded + segmented` — бинаризация и сегментация перед распознаванием.

Рекомендуется сравнивать все три варианта, так как качество может зависеть от конкретного изображения.

## 5.2 Интерпретация результатов

- Область `Original` показывает исходное изображение, выбранное пользователем.
- Области `Baseline`, `Baseline + segmented` и `Thresholded + segmented` отображают результаты разных способов обработки и распознавания.
- По этим областям можно визуально оценить, какой вариант лучше сохранил структуру рукописного текста.

- Нижняя часть окна предназначена для вывода текстового результата и графика эмоциональной окраски.
- До запуска анализа в этой области отображается сообщение о том, что анализ эмоций недоступен.
- После обработки изображения программа показывает распознанный текст, исправленный текст, доминирующую эмоцию и распределение по эмоциональным категориям, если анализ применим к выбранному языку.

### 5.3 Рекомендации по использованию

- Для получения более качественного результата рекомендуется использовать изображения с четким рукописным текстом, без сильного размытия, бликов, резких теней и лишних объектов на фоне.
- Желательно, чтобы строка текста была расположена горизонтально, а буквы были достаточно крупными и контрастными относительно бумаги.
- Если результат распознавания оказался неточным, рекомендуется попробовать другой вариант обработки, изменить параметры `Mask threshold` и `Min area`, а затем повторно нажать `Analyze`.
- Для длинных строк и фраз часто полезны варианты с сегментацией, так как программа распознает отдельные фрагменты изображения и затем объединяет результат.
- Анализ эмоциональной окраски предназначен прежде всего для русского текста. Он работает надежнее, если во входном изображении содержится не одно короткое слово, а фраза из нескольких содержательных слов.
- При слишком коротком тексте или большом количестве ошибок OCR эмоциональный анализ может не выявить доминирующую эмоцию.
- Для завершения работы с программой достаточно закрыть окно приложения стандартной кнопкой закрытия окна.

## 6 Рекомендации программисту и дополнительные сведения о ПО

- Программа написана на языке Python с использованием библиотеки PyTorch для нейросетевой части, Numpy для обработки данных и других вспомогательных библиотек.
- Исходный код блоков по обучению и инференсу CRNN-модели, а также предобработки изображений и анализа эмоций, находится в папке `src`. Рекомендуется ознакомиться с кодом, чтобы понять структуру программы и логику работы каждого компонента.
- Для удобства навигации по коду также создан Jupyter Notebook `train.ipynb`, который содержит код для обучения модели и может быть запущен в среде Jupyter.

- При запуске программы необходимо убедиться, что все зависимости установлены, и что выбранные checkpoint-файлы соответствуют языку распознаваемого текста. Рекомендуется использовать предоставленные checkpoint-файлы, так как они были обучены на соответствующих датасетах и обеспечивают оптимальное качество распознавания.

- Системные требования для запуска программы включают наличие Python 3.8 или выше, а также достаточного объема оперативной памяти и видеокарты для работы с нейросетью. Рекомендуется использовать виртуальное окружение для управления зависимостями и обеспечения совместимости.

Работоспособность инференса на GPU была проверена на следующих системах:

- Windows 11, Python 3.14, PyTorch 2.11, Процессор AMD Ryzen 7 7840H, 16 Гигабайт ОЗУ, Видеокарта AMD Radeon 780M

- Windows 11, Python 3.14, PyTorch 2.11 CUDA 12.8, Процессор Ryzen 5 8400F, 32 Гигабайта ОЗУ, Видеокарта NVIDIA GeForce RTX 3060 12GB

- macOS 26.2, Python 3.14, PyTorch 2.11, SoC Apple M1 Pro, 16 Гигабайт ОЗУ

На всех перечисленных системах инференс работает корректно, а качество распознавания соответствует ожиданиям, что подтверждает универсальность и стабильность разработанной программы.

Запуск инференса на CPU также возможен, но может занять значительно больше времени, особенно для более сложных изображений и длинных строк текста. Рекомендуется использовать GPU для ускорения процесса распознавания.

Обучение производилось на видеокарте NVIDIA GeForce RTX 3060 12GB, и заняло примерно 2 часа для достижения приемлемого качества распознавания. Рекомендуется использовать аналогичное или более мощное оборудование для обучения модели, так как процесс может быть ресурсоемким.

Перед запуском обучения рекомендуется проверить установку последних версий драйверов видеокарты, а также убедиться что версия PyTorch поддерживает CUDA, если планируется использовать GPU для ускорения обучения.

Запуск обучения на CPU не рекомендуется, так как это может занять значительно больше времени (до нескольких дней), в зависимости от объема данных и сложности модели.

## Список литературы

[1] U. Marti and H. Bunke. The IAM-database: An English Sentence Database for Offline Handwriting Recognition. Int'l Journal on Document Analysis and Recognition, Volume 5, pages 39 - 46, 2002.

[2] Handwritten Kazakh and Russian (HKR) database for text recognition.